

ENGINEERING DESIGN NOTE · 工程设计文档

Specter：本地推理加速引擎

手写投机解码× AWQ量化×自适应步长控制器——面向Agent场景的本地推理加速工程设计

版本

v9

日期

2026-08-27

状态

计划稿· 等待用户确认后进入阶段0

§0-1 / VERSION & ABSTRACT

版本信息与摘要

用户要求以资深架构师视角，从可行性和是否已有成熟方案两个角度评审v8。评审基于实证调研，发现一处实验设计问题、一处工作量估计偏乐观，以及若干未被计划采用的现成方案。v9据此修改了相关章节，架构方向和支柱1-5的整体设计保持不变；正文中标注“v9新增”或“v9修正”的段落是本版改动落点。

版本迭代记录

版本	内容
v1	三支柱构想，基于摘要级调研
v2	精读论文原文后修正三处理论风险
v3	生产复盘后新增支柱4（batch size交叉点）
v4	发现平台兼容性硬约束，新增支柱5（自适应控制器），补充12个具体已知坑
v5	按工程设计文档骨架重组，补齐目标/非目标、考虑过的其他方案、依赖假设、里程碑判据、悬而未决问题
v6	对标参考文档前18页的密度：三层指标体系、机制级拆分（P1.0-P5.4）、端到端走读、叙事收尾、三份附录
v7	读完参考文档全部28页后补四处颗粒度缺口：走读加入失败/恢复场景、公式配数值算例、真实文献列表内嵌、附录C加可执行诊断代码；新增“研究诚信护栏”一节
v8	精读5篇自适应控制论文原文，更正一处错误引用（SpecKV “41.2%” 实为误归因，真实数字2→8/4倍），重新措辞BanditSpec “太复杂” 的不准确判断，补充若干可精确溯源的新引用
v9	资深架构师视角评审：修正P5.2实验方法论错配风险、AWQ工作量估计上调、补齐三处“已有成熟方案未利用”的信息差、新增self-speculative备选方案讨论、新增云端预算本地dry-run硬性约束
v9追加	结合求职定位（SWE/Agent交叉岗位而非研究产出）重新评审文献重叠：新增5篇论文精读（QSpec/Synergy/Learning to Draft等），把“填补空白”类措辞统一改写为“复现验证”，更正GammaTune作者引用错误，新增P3.2最小服务层封装+agent循环端到端演示

摘要

Specter是一个面向Agent场景的本地推理加速引擎：手写实现投机解码（含严格正确性证明）和AWQ风格量化，用一套系统原理（内存带宽优化vs计算瓶颈切换点）统一两者的失效边界，再加一个训练-free的自适应投机步长控制器，并封装成可被agent循环实际调用的最小服务接口（v9追加，见P3.2）。目标硬件：个人24GB统一内存Mac（开发/调试，\$0）+短期云端GPU租用（规模化验证，\$30-50）。当前状态：计划已经过三轮论文精读（含一轮针对5篇自适应控制论文、一轮针对文献重叠程度的10篇补充精读）、一轮生产实践调研、一轮针对性踩坑调研、一轮资深架构师可行性评审，尚未开始任何代码实现。

§2 / BACKGROUND

背景与问题陈述

2.1为什么现在做这件事有意义（不只是“学新技术”）

一个残酷的招聘现状.来自我自己整理的《2026届北美CS/SWE招聘市场研究》：NACE 2026年春季雇主调查（2026-02-12至03-17，n=185）里，43.4%的雇主表示已经检测到求职材料是AI生成的；同期SignalFire《2026科技人才状况报告》（2026-06-22）显示，大型科技公司应届生/入门级岗位招聘量比2019年下降约65%；Stanford数字经济

实验室基于ADP薪资数据（覆盖至2025年中/末）发现22-25岁软件开发者就业相比2022年末峰值下降约20%。当AI能替任何人生成一份看起来专业的项目描述、而入门级岗位本身又在收缩时，简历上的一行bullet本身已经不构成信号。真正稀缺的是可验证的技术判断力痕迹——你在实现之前有没有读懂边界条件、有没有踩过真实的坑、有没有诚实地说清楚“我为什么没选那个更优的方案”。

Specter的定位是一份留下判断痕迹的工程记录：12个具体的、有来源的已知坑，8处“考虑过但拒绝”的替代方案及其理由，以及一个从平台兼容性调研中反推出来的架构决定。这些内容AI可以帮忙写代码，但编不出“调研后发现vLLM在Mac上跑不动，因此把方案拆成两个阶段”这类只有真正做过调研才会有具体细节。

差异化的诚实边界（v9修正：从“填空白”改为“复现+集成”）

GitHub上已经有多个成熟的“手写投机解码”公开实现（romsto/Speculative-Decoding、suryavanshi/speculative_decoding、dilab-zju/self-speculative-decoding等），单纯“我手写了投机解码”这句话本身，作为简历差异化程度已经不高。v9修正：量化-投机步长耦合（QSpec, arXiv 2410.11305）、batch交叉点的系统性刻画（Synergy of SD and Batching, arXiv 2310.18813）、compression-aware的自适应控制

（SpecKV）、batch-aware熔断（Nightjar）这几件事分别已经有论文做过，不能再说“文献里没人测过”。这个项目的差异化不在于研究结论新，在于把这几条独立结论在一个人力所及的范围内亲自复现+验证+串成一个能跑的本地系统，并且诚实记录复现过程中和原论文结论不一致的地方——这本身对求职（尤其SWE/Agent交叉岗位）比“发现新现象”更容易被面试官验证，因为可复核。

2.2 技术现状与缺口

投机解码和量化在2025-2026已经是主流推理框架的标配（vLLM/SGLang一个flag就能开）——这意味着“会调库”本身已经没有技术含量。v9修正：以下三点原先写作“缺口”，经核查后更准确的说法是“可复现验证的具体问题”，而不是“没人做过”——(a)手写实现能否达到和调库一致的正确性保证（这一点确实少有人公开验证，仍是真实缺口）；(b)这些技巧在什么条件下失效（batch size、压缩程度）——QSpec、SpecKV等论文已经系统测过，这个项目做的是在AWQ这一个具体量化方法+自己的硬件约束下复现同类实验，不是补空白；(c)主流生产库还没内置的“自适应控制”能做到什么程度——Nightjar已经把batch-aware熔断做到生产级，这个项目做的是用更轻量的实现复现类似效果，不是原创设计。三点合起来，这个项目要做的是“亲手把这些已发表结论在自己的约束下验证一遍、并集成成一个可用系统”，不是“调库拿不到的部分”。

§3 / GOALS & NON-GOALS

目标与非目标

目标

- 1. 手写实现投机解码核心采样算法，贪心模式逐token精确验证正确性，采样模式统计层面验证分布等价。
- 2. 手写实现AWQ风格激活感知量化校准，诚实区分“自己写的算法”和“复用的现成kernel”。
- 3. 用统一系统原理（内存带宽vs计算瓶颈）解释投机解码和量化各自的失效边界，实测出具体交叉点。
- 4. 实现训练-free自适应投机步长控制器，验证其在压缩模型和生产基线（HF Transformers内置动态投机）面前的表现。
- 5. 用AgentBench方法论改编的agent工具调用评测集，验证结构化输出场景下投机解码的优势。
- 6. 把加速后端包装成可被agent循环实际调用的最小服务接口，跑通一次多轮工具调用的端到端延迟对比演示（v9新增，见§7 P3.2）——离线跑分证明“方法有效”，这一条证明“能接进真实系统”。

非目标（显式声明，防止范围蔓延）

- 1. 不重新实现GPTQ——太复杂，用LLM Compressor做对比基线。BitsAndBytes NF4例外：HF Transformers内置几行配置即可开启，不算“重新实现”，P5.2的对照臂会用到（见坑10）。
- 2. 不做完整AgentBench八个环境——只改编OS/bash子环境。
- 3. 不重新实现BanditSpec的bandit理论方法或AdaEDL的熵基提前停止——只作对比引用；BanditSpec有公开代码（github.com/sail-sg/BanditSpec），P5.4允许直接跑这份代码做基线对比，但不允许把它的算法搬进自己的P5实现里。
- 4. 不训练专用speculative head（EAGLE/Medusa那种）——没有多卡训练资源，经典两模型方案是预算约束下的合理取舍。

- 5.不追求在Mac上跑出真实速度数字——但量化速度例外：mlx-lm有原生AWQ支持（mlx_lm.awq），可以在Mac上产出真实的、Metal加速的AWQ速度数字，见§7 P2.4；此例外仅限“借用现成工具产出一个交叉验证数字”，不改变“投机解码真实速度只能在云端测”这条主约束。
- 6.不做多租户/高并发服务化——关注单用户/低batch的本地agent场景。
- 7.不采用self-speculative/layer-skip架构（v9新增，见§8）——两模型架构本身是项目故事的核心，self-speculative会规避掉正想展示排查能力的那类失效模式（tokenizer不一致、bonus token跨模型采样）。

§4 / TENETS

指导原则（Tenets）

1.能在免费环境验证的，绝不烧云端预算

决定Mac/云端两阶段分工；云端阶段的脚本必须先在本地图极小dummy模型跑通全流程再上云（v9新增，见§10）。

2.诚实边界优于假装SOTA

每处方案选择都要写清楚“生产界最优解是什么、我们为什么没选它”，给出的理由应是实际的技术判断依据。

3.正确性验证要有可复现的数学/统计依据

贪心逐token精确比对、采样统计检验+任务指标双重验证。

4.每个技巧不仅要证明“有效”，还要测出“什么时候失效”

batch size交叉点实验和量化-步长耦合实验的存在理由。

5.基线出来之前不编造目标数字

（借用Time x Mart文档的原则）第5节的所有阈值都标注“预期区间”而非“承诺值”，实际数字以阶段0-6跑出来的结果为准。

6.测量本身也要接受怀疑

一个“符合预期”的实验结果不会自动免检；必须先证明测量工具本身是可靠的（测试的测试），再采信结果。

7.引用别人的数字之前，先确认自己真的读对了

转述/摘要会丢失或扭曲归因关系；任何要写进计划或最终报告的具体数字，引用前必须能标注清楚出处章节、表号和归属。

8.对标一个实验结果之前，先确认自己的方法和对方是不是同一件事（v9新增）

不同的量化方法/实验设置测出来的现象，即使“名字一样”，底层机制也可能不同；直接对标数字而不核对方法论前提，会把方法差异误判为实验失败（P5.2的AWQ/BnB错配是触发案例）。

9.别人已经做过的部分，优先用现成工具交叉验证（v9新增）

mlx-lm的原生AWQ、BanditSpec的公开代码都是可以直接拿来当参照系的现成资源，用好这些能省下时间和预算去打磨差异化的部分。

§5 / METRICS

指标体系（北极星/伴随/护栏）

沿用Time x Mart文档§6的三层指标框架。以下数字均来自自己读论文的原始实验结果，代表“这个方法在别人的实验里做到过什么量级”，并非Specter自身的实测承诺——Specter自己跑出来的数字以最终README为准，可能高于也可能低于这些参考区间，这正是实验要回答的问题。

北极星指标 · 正确性优先于速度

贪心模式下投机解码输出与目标模型直接推理的token级一致率——目标100%（允许可忽略的浮点误差），这是唯一不允许“这次没做到但下次再说”的指标。

伴随指标（每个支柱一个，参考区间来自文献）

指标	参考区间（来自文献，非承诺值）	来源
投机解码接受率 α	≥ 0.65 才有净加速（低于则收益递减）	mlx-lm生产实践阈值
投机解码整体加速比	2-3x（生产环境实测）	IBM/PyTorch官方生产博客
4-bit量化显存降低	约70%，perplexity涨幅 $< 5\%$ （同分布校准）	AWQ论文
跨分布校准perplexity涨幅差异	AWQ +0.5-0.6 vs GPTQ +2.3-4.9	AWQ论文实验
GammaTune风格自适应控制器vs 固定 γ	$15\% \pm 5\%$	GammaTune论文 SpecBench结果
量化程度对最优 γ 的偏移幅度	SpecKV在BnB压缩下最高4倍（FP16下 $\gamma=2 \rightarrow$ INT8下 $\gamma=8$ ）；AWQ下的偏移幅度是本项目要回答的开放问题，不预设应复现同等量级（v9更正，见坑10）	SpecKV论文Table 1
高负载下投机解码相对无投机的倒退幅度	最高30.25%	Nightjar论文生产实测（RTX 4090, DeepSeek-R1-Distill-Qwen-7B, ShareGPT）

护栏指标（不能突破的底线）

- 采样模式下游任务指标差距（HumanEval pass@1 / ROUGE）必须 < 2 个点，否则视为“分布等价”验证失败，需要回头检查采样实现而不是接受结果。
- 云端预算硬顶\$50，超支即停止规模化验证，用已有结果收尾，不追加开支。
- 任何“自适应控制器让情况变得更差”的场景（GammaTune论文承认的边界）必须被记录进报告，不能因为不好看就不提——诚实边界本身是护栏。

§ 6 / LANDSCAPE

现有方案与文献综述

条件性结论（完整分类文献列表见附录E，不再用外链指针代替内容）：投机解码/量化已是主流框架标配，真正区分度在手写实现+正确性验证+边界刻画；IBM/PyTorch生产复盘证实代码/结构化输出场景效果最好，batch > 64 开始吞吐下降；AWQ相对GPTQ对校准集分布更鲁棒；2024-2026自适应投机控制是活跃前沿方向（至少7-8篇独立工作，v9评审时新发现的AdaSD是又一例，方向不重叠于P5.2，见附录E）。v9再次评审补充：这个方向比之前判断的更拥挤——QSpec（arXiv 2410.11305）已经画过AWQ $\times$ batch size的加速比收窄曲线（对应支柱4想做的实验），Nightjar（arXiv 2512.22420）已经在vLLM上做出了batch-aware熔断+重探测的生产级实现（对应P5.3），本项目对这两部分的定位相应从“填空白”调整为“在自己的约束下复现验证”（见§2.2、P5.3）。动态 γ 调整已是HF Transformers默认行为（4.45.0起，且v9评审发现该默认行为其实包含两套不同机制，见§7 P5.4）。

§ 7 / ARCHITECTURE

方案与架构（机制级拆分，对标R0-R17的颗粒度）

REFERENCE

Mac开发阶段（阶段0-4，\$0）：HF Transformers（PyTorch, MPS backend）
云端GPU阶段（阶段5-6，\$30-50）：vLLM对比基线+ LLM Compressor

支柱1：手写投机解码+正确性验证

P1.0—前置存活性gate（S，半天）

断言`draft_tokenizer.get_vocab() == target_tokenizer.get_vocab()`；测 α ，两级门槛： <0.4 换模型对； $0.4-0.65$ 降低预期继续； ≥ 0.65 正常推进。用`mlx-lm`跑同一模型对交叉验证 α 数量级。

P1.1—Rejection Sampling核心算法（M，2-3天）

标准算法：草稿模型自回归生成 γ 个候选token，目标模型单次前向对全部候选打分；接受判据 $p_{TM}(x) \geq p_{DM}(x)$ 时接受，否则以 $1 - p_{TM}(x)/p_{DM}(x)$ 概率拒绝，从调整分布 $p'_{TM}(x) = \text{norm}(\max(0, p_{TM}(x) - p_{DM}(x)))$ 重采样。关键实现检查点：bonus token必须来自目标模型分布，不是草稿模型（坑2）。数值算例见附录A.1。

P1.2—贪心模式验证器（S，1天）

逐token严格比对投机解码输出vs目标模型直接推理输出，要求100%一致（或仅浮点误差）。

P1.3—采样模式验证器（M，2天）

统计层面验证实测 α 与理论公式 $\alpha = E[\min(p, q)]$ 吻合；下游任务指标parity（HumanEval pass@1 / ROUGE，差距 <2 点）。

P1.4— γ 扫描（S，1天）

$\gamma \in \{1, 3, 5, 7, 10\}$ ，记录接受长度分布（预期形状参考AdaEDL论文Figure 1 / Appendix Fig 7c：Dolly-15k创作数据集，目标模型Llama2-7B，草稿模型是对齐微调过的115M模型，不同DL下方差递增，DL=3 $\rightarrow$ std ≈ 1.2 ，DL=7 $\rightarrow$ std ≈ 1.92 ，DL=16 $\rightarrow$ std ≈ 2.35 ，这是“为什么需要自适应 γ ”的实证基础，支柱5会复现类似分布）。

支柱2：AWQ量化

P2.0—激活统计采集（S，1天）

在校准集上跑前向，收集每层激活值的逐通道统计量。

P2.1—逐通道缩放校准算法（M，3-4天，v9调整：原估计2天偏乐观）

AWQ核心思想：找到一组逐通道缩放因子，使量化后误差最小化，同时保护“显著性权重通道”（由激活值幅度决定）不被粗暴量化。直觉性数值算例见附录A.3。工作量修正说明：实现难点主要在于fake-quantize模拟（round $\rightarrow$ clamp $\rightarrow$ dequantize）的实现顺序调试，而非缩放因子的数学步骤本身——顺序或数值范围出错会导致perplexity大幅偏离预期，且往往需要逐层排查才能定位问题所在，加上激活值统计采集时机（hook挂在模块输入还是输出）和缩放因子网格搜索范围也需要反复试验。2天足以写出初版实现，3-4天是让perplexity数字可信的现实估计。

P2.2—跨分布鲁棒性实验（S，1天，坑5）

两个分布明显不同的校准集（代码语料vs自然语言）分别校准，交叉在两种评测集上测perplexity，验证“AWQ跨分布涨幅远小于GPTQ”这一AWQ论文核心论点是否在自己的模型上复现。

P2.3—校准集大小消融（S，半天，坑6）

“校准集大小vs perplexity”曲线，验证过小样本（几十条量级）是否出现过拟合迹象。

P2.4—速度对比（M，v9调整：Mac可先出一部分，云端补全）

云端阶段用LLM Compressor跑GPTQ基线，测真实加速比——这个数字Mac阶段无法产出（平台约束，具体技术原因见§9.1风险B：PyTorch旧版量化API从未移植到MPS后端）。v9新增：Mac阶段可以先用`mlx_lm.awq`（`mlx-lm`原生支持的AWQ量化路径，不是自己写的实现）跑一个真实的、Metal加速的Mac端AWQ速度/内存数字，作为免费的交叉验证点——不能替代云端阶段和GPTQ的对比，但能在花云端预算之前先建立一个“AWQ大概应该长什么样”的参照系。

支柱3：AgentBench-OS子集评测

P3.0—任务集改编（M，2天）

15-20个bash/工具调用任务，改编自AgentBench OS子环境，具体任务清单见附录B。

P3.1—接受率对比实验（S，1天）

同一模型对，工具调用/结构化输出场景vs自由文本对话场景，对比 α 差异，验证“结构化输出接受率更高”这一生产界共识。

P3.2—最小服务层封装+ Agent循环端到端演示（M，2天，v9新增）

P3.0/P3.1之前只是离线跑分（脚本直接调用模型对象），面试官看不到“这东西能不能被一个真实agent系统调用

”。这里补一层最小HTTP服务（FastAPI或标准库http.server均可，不追求生产级鲁棒性）：单一/generate端点接受prompt+工具定义，内部路由到“投机解码+AWQ+自适应 γ ”加速后端或“目标模型直接推理”基线后端，两者共享同一接口。演示脚本跑一个多轮工具调用任务（复用P3.0的任务子集，模拟agent循环反复调用该端点、解析返回的工具调用、把结果拼回下一轮prompt），记录端到端墙钟延迟（不是单次前向延迟）在加速后端vs基线后端下的对比。退出判据：能用一条curl/Python客户端命令跑通完整多轮流程并输出延迟对比表，不要求支持并发或长期运行。这一条不改变支柱3的评测结论本身，只是把“接受率数字”包上一层“确实可以被调用”的验证——对应SWE/Agent交叉岗位实际会问的“这个能接进我的系统吗”，而不是“这篇论文的结论对不对”。

支柱4：Batch Size交叉点实验

P4.0—Mac小模型扫描（S，1天）

batch $\in \{1, 4, 8, 16, 32, 64\}$ ，7B级别模型，投机解码和量化各测一条吞吐提升曲线。

P4.1—显存占用记录（S，与P4.0同步）

每个batch size下草稿模型权重和KV cache各自估算显存占用（坑8）。

P4.2—云端大模型验证（M，云端阶段）

更大模型/更高batch验证交叉点是否随模型尺寸偏移，对标Nightjar 30.25%倒退量级（坑7）。

支柱5：自适应投机步长控制器

P5.0—GammaTune算法实现（M，2天）

Algorithm 1：接受数A与当前 γ 相等时扩窗（ $\gamma \leftarrow A + \delta$ ）；否则EMA更新（ $\gamma^- \leftarrow \text{clip}(\gamma_{\min}, \gamma_{\max}, (1-\eta)\gamma^- + \eta A)$ ， $\gamma \leftarrow \gamma^- \gamma$ ）。三轮数值算例见附录A.2。

P5.1—波动场景鲁棒性测试（S，1天，坑9）

混合代码生成+开放聊天的prompt序列，中途切换任务类型，验证GammaTune在非稳态场景下是否还有优势，还是像论文承认的那样收益打折。

P5.2—量化- γ 耦合实验（M，云端阶段，坑10，v9重新设计）

复用P2的量化模型（FP16 vs自己的4-bit AWQ版本），对同一draft/target组合分别扫描最优 γ 。v9修正：SpecKV的2 $\rightarrow$ 8偏移是在BitsAndBytes压缩下测出来的，AWQ的逐通道缩放本身是为了保护输出分布不被大幅扰动而设计的，两者不是同一件事，直接拿AWQ的结果对标BnB测出的具体倍数存在方法论错配风险。因此：(a)增加一个可选的BitsAndBytes NF4对照臂（HF Transformers内置load_in_4bit=True几行配置，不算重新实现GPTQ），直接复现SpecKV的原始设置作为“同源对照”；(b)自己AWQ模型的实验成功判据从“复现4倍量级”改写为“验证方向一致（压缩程度越高最优 γ 越小）”即可，具体偏移量级本身就是可报告的发现”。

P5.3—Batch-aware熔断器（M，2天，坑11，v9重新定位：明确为复现而非原创设计）

v9修正：这个机制本质上是Nightjar（arXiv 2512.22420）的简化复现，不是原创设计——Nightjar已经用多臂老虎机（MAB）规划器在vLLM上做出了“检测到投机解码不划算就禁用+卸载草稿模型、负载降下来后台重新加载”的生产级实现。这里用一个更轻量的周期性定时器代替它的MAB规划器：高batch时自动降级/禁用投机解码；必须包含周期性重探测机制（避免DSD式“重激活难题”）；必须实测切换开销（KV cache重建成本，避免BanditSpec式“假设切换免费”的问题；Nightjar实测同类切换开销量级为17.87-102.03ms，随输入长度/batch size变化，可作为自己实测数字是否合理的参考量级）。要报告的结论是“更简单的定时器方案能在多大程度上接近MAB方案的效果”，不是“我设计了一个新机制”。

P5.4—生产基线对比（M，1-2天，v9拆分为三个明确基线）

不再笼统写“对标HF内置”，明确对比三个基线：(a) HF Transformers num_assistant_tokens_schedule="heuristic"（简单启发式，4.45.0起默认）；(b) HF Transformers assistant_confidence_threshold（基于置信度阈值的无监督自适应机制，是Intel/HF“Dynamic Speculation Lookahead”论文的落地版本，官方数据显示最高2.7x提速，方法论上比(a)更接近AdaEDL的熵/置信度思路，是更强的基线）；(c) v9新增：BanditSpec公开代码（github.com/sail-sg/BanditSpec，兼容LLaMA/Qwen2架构）直接克隆跑一遍作为第三个对比点——用现成代码做对比，不是重新实现它的bandit理论。给出定量对比结论（不要求超越，要求诚实数字）。

§ 8 / ALTERNATIVES CONSIDERED

考虑过的其他方案

九处关键架构/方案决策点及其拒绝的替代方案，理由需能追溯到具体技术约束或项目目标。

决策点	选择的方案	考虑过的替代方案	拒绝理由
投机解码架构	经典独立草稿模型	训练专用speculative head (EAGLE/Medusa)	需要多卡训练资源（IBM用FSDP两阶段训练），个人项目没有这个预算
投机解码架构	经典独立草稿模型	self-speculative decoding (LayerSkip/Draft & Verify等，v9新增)——同一个模型跳过中间层生成草稿，不需要第二个模型	不需要第二个模型意味着天然规避tokenizer不一致（坑1）和bonus token跨模型采样（坑2）这两类失效模式，但这两类失效模式正是本项目想展示排查能力的内容；采用self-speculative会使“两模型架构”这条串联量化与投机解码的主线失去意义。拒绝原因是它会规避掉项目想展示的失效模式，与实现难度无关
量化自研对象	AWQ	GPTQ	实现复杂度（Hessian/Cholesky分解）和时间线不匹配；AWQ对校准集分布更鲁棒
量化对比基线工具	LLM Compressor（云端）+ mlx_lm.awq（Mac，v9新增）	分别装 AutoGPTQ+AutoAWQ	行业已转向统一工具；AutoGPTQ/AutoAWQ在Mac上都不可用；mlx_lm.awq可以在Mac上免费产出一个真实的AWQ速度交叉验证点，不必等到云端阶段
Mac开发阶段主框架	HF Transformers (MPS backend)	(a) vLLM CPU (b) vLLM-metal (c) MLX主实现	(a)比llama.cpp慢20-30倍(b) 2026年仍是早期版本 (c) HF Transformers有论文先例（SpecKV方法论），MLX降级为交叉验证工具
Agent评测范围	AgentBench OS子环境	完整八个环境	完整版对时间线太重；OS子环境恰好是投机解码效果最好的场景
支柱5控制器算法	GammaTune (EMA)	BanditSpec (bandit理论) /AdaEDL (熵基早停)	BanditSpec的UCBSpec核心算法本身并不复杂（论文自称是最简单的一类UCB算法，且有公开代码可直接跑做对比），未采用的原因是它的核心目标（regret-bound理论保证）和AdaEDL的核心目标（熵阈值提前停止）与Specter支柱5的核心目标（自适应步长）只有部分重叠，引入完整框架的抽象成本和项目目标不成比例；两者保留为文献对比对象，BanditSpec另可直接跑其公开代码做基线对比
量化- γ 耦合实验的压缩方式	AWQ（主）+ BitsAndBytes NF4（对照，v9新增）	只用AWQ单一压缩方式	只用AWQ会导致对标SpecKV数字时出现方法论错配（AWQ和BnB对输出分布的扰动机制不同）；加一个BnB对照臂几乎零成本（HF内置配置），能直接复现SpecKV的原始设置作为同源参照
正确性验证标准	贪心逐token严格比对+采样统计检验	对采样模式也要求KL散度严格为0	不同设备/框架间不保证bit-exact输出，严格要求KL=0不现实且无必要

§ 9 / KNOWN RISKS

已知风险与应对

9.1 平台兼容性硬约束

风险A：vLLM在Mac上不能用

vLLM在Mac上原生不可用（vLLM Issue #1441实测报错"Cannot find CUDA_HOME"）；v9二次核查更正：此前版本引用的"CPU模式3-5 tokens/s vs llama.cpp Metal 92 tokens/s"具体数字经查证实际不在Issue #1441正文或115条评论里（逐条搜索确认），属于引用溯源错误——数字量级（20-30倍差距）在多篇第三方2026年评测里独立吻合，可信，但不应挂在这个issue名下，此处改为不依赖精确数字的定性表述。社区维护的vllm-metal插件2026年内进展较快（v0.2.0起统一Metal kernel、已用上M5 NAX张量单元），但走的是MLX技术栈而非vLLM官方CUDA路径，仍不能替代云端阶段用真正vLLM做基线对比。应对：Mac阶段用HF Transformers手写实现（SpecKV论文方法论原文如此），云端阶段才引入vLLM做对比基线。

风险B：AutoAWQ/AutoGPTQ/LLM Compressor全部要求CUDA

AutoAWQ要求Compute Capability ≥ 7.5 , CUDA ≥ 11.8 ；AutoGPTQ维护者明确需要专门写MPS kernel（未实现）；LLM Compressor底层依赖和vLLM生态一致。v9补充更精确的技术原因：PyTorch的旧版量化API（torch.quantize_per_tensor等）用的是独立的“Quantized”后端，从未被移植到MPS，直接报错"QuantizedMPS"backend not implemented——这是Mac上做不了真实int4/int8张量运算的根本原因，不只是第三方库懒得适配。来源：AutoGPTQ Issue #223。应对：支柱2拆两半，Mac阶段只产出正确性/压缩率数字（fake-quantize模拟，普通float运算，不受此限制），云端阶段才测真实速度；例外见P2.4，mlx-lm走的是自己的Metal kernel路径，不受PyTorch这个限制。

参照系：MLX生态经验阈值

mlx-lm文档：接受率 $\alpha > 0.65$ 才有净加速。应对：阶段0 α -gate改两级判断（见P1.0）。

9.2 支柱1已知坑

坑1

tokenizer/vocab不一致使接受率静默归零，不报错。同一模型家族内部也可能不一致（Qwen2 1.5B vocab_size=151936, 72B=152064）。应对：P1.0显式断言检查。

坑2

bonus token采样源搞错——已知真实bug（DSD）：错误地从草稿模型分布采样bonus token，违反正确性保证但不会崩溃。应对：单元测试专门检查bonus token采样代码路径来自哪个模型的logits。

坑3

batch>1时ragged tensor同步问题——不同序列接受的草稿token数不同，position id/attention mask/KV cache长度参差不齐。应对：支柱4设计里手动维护per-sequence状态，不依赖框架自动padding。

坑4

draft/target尺寸“死亡区间”——尺寸差距不够大（2-3倍）可能比不用还慢。应对：诊断标准—— α 尚可但墙钟时间变慢，先查草稿模型自身延迟占比。独立佐证：AdaEDL论文的对照实验里，用未经微调的TinyLlama-1B给Llama2-7B做草稿模型、固定投机长度=7时，静态投机解码（Base-SPD）反而比不用投机解码的自回归基线慢16%；而同样的模型对，只要加上自适应提前停止（AdaEDL/Max-Confidence-SPD），就变成比自回归快43%——同一组模型，“静态步长”和“自适应步长”的差别决定了投机解码是帮倒忙还是真加速，这是坑4现象的一个独立数据点。

9.3 支柱2已知坑

坑5

校准集分布不匹配使GPTQ严重过拟合，AWQ相对稳健——跨分布（PubMed→Enron）时AWQ涨0.5-0.6，GPTQ涨2.3-4.9。应对：P2.2交叉验证实验复现这个矩阵。

坑6

校准集太小会过拟合——标准实践C4数据集、group size 128。应对：P2.3做校准集大小消融。

9.4 支柱4已知坑

坑7

Nightjar实测高负载下投机解码相对无投机倒退最多30.25%，比“batch>64开始下降”更极端更具体。应对：P4.2对标这个数字。

坑8

草稿模型权重和KV cache抢显存，独立于计算瓶颈之外的第二个原因——与我们自己24GB Mac的资源约束是同一原理的缩影。应对：P4.1记录显存占用维度。

9.5支柱5已知坑

坑9

GammaTune自己承认的边界——draft/target匹配度高（方差小）时收益有限，依赖历史接受率使其在对抗性/剧烈波动场景下退化。应对：P5.1设计波动场景测试。

坑10（v9补充方法论注记）

SpecKV自己的实验（Table 1）发现最优 γ 随目标模型压缩程度（BitsAndBytes的FP16/INT8/NF4，不是AWQ）从FP16下的2偏移到INT8下的8，偏移量级4倍——量化和自适应控制不是独立技巧。v9新增方法论注记：这个4倍数字是在BnB压缩下测出来的，AWQ的逐通道缩放设计目的是最小化量化对输出分布的扰动，两者机制不同，直接拿自己的AWQ模型去对标这个具体倍数存在方法论错配风险——如果AWQ下测出的偏移明显小于4倍，可以将其视为“AWQ对投机解码更友好”的一个值得报告的发现，无需归结为实验错误。应对：P5.2复用支柱2的量化模型验证偏移方向；同时加一个BitsAndBytes NF4对照臂直接复现SpecKV原始设置作为同源参照。

坑11（v9再次核查：P5.3与Nightjar的关系需要重新定位）

Nightjar批评DSD“禁用后不再收集观测数据，难以重新启用”，批评BanditSpec“忽视KV cache重建开销”，并且已经用MAB规划器把“检测不划算就禁用+后台重新加载”这套机制做成了vLLM上的生产级实现。P5.3不是在填Nightjar留下的空白，而是它的简化复现：用周期性定时器代替MAB规划器，验证更轻量的方案能否达到接近的效果。应对：P5.3熔断器必须含周期性重探测机制+实测切换开销；Nightjar自己实测的同类开销在RTX 4090+DeepSeek-R1-Distill-Qwen-7B配置下是17.87ms（短输入/小batch）到102.03ms（长输入/大batch），可作为自己实测数字量级是否合理的参照。

坑12（v9再次更新）

BanditSpec论文自己在附录B.2说UCBSpec是“最简单的一类UCB算法之一”（只维护经验均值+置信半径），因此不复现其完整bandit理论的原因并非实现复杂度，而是：(a)它的K-armed框架没有把batch size当作决策的上下文特征；(b)它的regret-bound理论分析没有建模KV cache重建的切换成本，这正是P5.3要弥补的地方。v9补充：BanditSpec有公开代码，可以直接克隆跑起来做P5.4的第三个对比基线，省去重新实现的时间。应对：只作对比引用，不重新实现算法本身；P5.4直接跑其公开代码做基线；P5.3的切换开销实测吸收了这个批评里最相关的部分。

9.6研究诚信：防止自己欺骗自己

参考文档有一份“风险台账与反刷设计”，防止平台被用户系统性钻空子。Specter没有外部用户，对应的风险是评测/调参过程中无意识地让结果看起来比实际更好——这在没有同行评审、只有自己既是研究者又是评审者的个人项目里尤其容易发生，需要显式设计对策来应对，不能假设自己足够客观。

风险1：评测集/校准集泄漏到调参过程

如果P5.0的GammaTune超参数（ $\eta, \delta, \gamma_{\min}/\max$ ）是在P3.0的AgentBench-OS任务集上反复试出来的，那么P5.1/P5.4用同一套任务集“验证”控制器有效，验证的只是过拟合。应对：P3.0的15-20个任务在设计阶段就固定切出一部分（建议3-5个）作为held-out set，只在所有超参数定下来之后跑一次，不回头调整。

风险2：Best-of-N挑好看的一次运行报告

batch扫描（P4）、 γ 扫描（P1.4）、量化对比（P2.4）这类实验都有随机性（采样温度、校准集shuffle顺序），如果每个数字都是“跑了几次挑最好的一次”，报告出来的加速比会系统性偏高。应对：凡是进最终报告的数字，至少跑3次取均值±标准差；如果标准差大到影响结论（比如GammaTune 15%提速的置信区间跨过了0），如实写出“这次没有统计显著性”，而不是只报均值。

风险3：正确性验证器本身有bug却一直“通过”

P1.2的贪心比对器如果实现有误（比如比较逻辑写反、或者两条推理路径共享了同一份缓存导致“殊途同归”式的假阳性），会一直静默通过，给人“正确性已验证”的错觉，比不验证更危险。应对：故意注入已知会破坏正确性的bug（比如手动让bonus token从错误的模型采样、或者手动打乱一个draft token），确认验证器必须报错——“测试验证器本身”是P1.2完成判据的一部分，不是事后可选项。

风险4：符合预期的结果比违反预期的结果少一道复核

如果P2.2真的复现出“AWQ比GPTQ更抗跨分布过拟合”，因为这和AWQ论文的结论一致，很容易不假思索地直接采信；但如果结果恰好和论文数字接近到可疑的程度，反而应该多查一遍。应对：任何“和文献高度吻合”的结果，都过一遍“如果这是bug造成的假象，最可能是哪个环节”的反向检查清单，再写进报告。

风险5：引用链条比原文长两层以上时，归因关系会悄悄失真

多手转述是最不容易被自己发现的错误类型，因为读起来完全通顺、不会触发“这里有问题”的直觉。SpecKV的41.2%误归因就是这样进入v6/v7的。应对：任何要写进计划或最终报告、且来自“精读论文”这个动作的具体数字，标注时至少包含“论文名+第几节/哪张表+是否为该论文自己的结果”三项，缺一项就视为未完成溯源，不能直接采信。

风险6：拿别人的实验结果当对标时，没核对方法论前提是否一致（v9新增）

风险5针对“数字本身记错了归属”，这条针对另一种更隐蔽的错误：数字归属没有记错，但拿来对标的自己的实验与原实验条件并不相同（P5.2的AWQ vs BnB就是这类风险的具体触发案例）。应对：任何“对标XX论文的YY数字”的表述，写之前先确认自己的实验设置和对方是否真的可比，不可比时要么补一个同源对照组，要么把预期改写成“方向应该一致，量级差异本身可以是发现”。

§10 / DEPENDENCIES & ASSUMPTIONS

依赖与假设

硬件/环境

- 开发机：Mac，24GB统一内存，无独立GPU
- 云端：Vast.ai/RunPod按需租用NVIDIA GPU实例

软件依赖（按阶段）

阶段	依赖	约束
Mac开发	HuggingFace Transformers, PyTorch (MPS)	确认所选模型对MPS算子支持完整；MPS不支持真实int4/int8量化张量运算（见§9.1风险B），量化正确性验证只能走fake-quantize模拟
Mac交叉验证	mlx-lm（含mlx_lm.awq原生AWQ路径，v9新增）	a数量级sanity check；mlx_lm.awq额外可产出真实的Mac端AWQ速度/内存数字，见P2.4
云端验证	vLLM	CUDA≥11.8
云端量化对比	LLM Compressor (GPTQ) + BitsAndBytes (NF4，v9新增，P5.2对照臂用)	CUDA环境，不支持Mac；BnB配置简单(load_in_4bit=True)，不算重新实现GPTQ
云端基线对比	BanditSpec公开代码 (github.com/sail-sg/BanditSpec，v9新增)	需要torch transformers fairscale flash-attn，flash-attn需针对CUDA/PyTorch版本编译；只用来跑对比，不修改其算法逻辑

模型/数据假设

- draft/target模型对暂定Qwen2.5系列，具体尺寸待P1.0结果确定
- 假设模型对共享一致tokenizer/vocab（P1.0显式验证，不假设）
- 量化校准：C4 +至少一个分布明显不同的第二数据集（P2.2）
- Agent评测：改编自AgentBench OS子环境的15-20个任务（附录B），其中3-5个held-out（9.6风险1）

云端预算执行纪律（v9新增）

所有计划在云端跑脚本（LLM Compressor校准、vLLM基线、BnB对照臂、BanditSpec代码、P4.2大模型batch扫描）必须先在本地图极小的dummy模型（比如2-4层的toy transformer，几秒钟能跑完一次完整流程）验证脚本逻辑无误，再上云端跑真实规模。理由：云端调试比本地调试烧钱快得多，\$50预算经不起“边跑边改bug”的消耗；云端时间只应该用来产出真实规模的数字，不用来定位代码逻辑错误。

§ 11 / ROADMAP

路线图与工作量（对标参考文档§6格式）

阶段、上线内容、工作量估计、所需环境/预算与完成判据的对应关系。

阶段	上线内容	工作量	环境/预算	完成判据
0	P1.0前置gate	S（0.5天）	Mac，\$0	vocab断言通过； $\alpha \geq 0.4$ ；mlx-lm交叉验证一致
1	P1.1-P1.4投机解码核心+验证	M（7天）	Mac，\$0	贪心100%一致；采样 α 与理论公式吻合；bonus token测试通过；验证器故障注入测试通过（9.6风险3）； γ 扫描曲线产出
2	P2.0-P2.3 AWQ校准+正确性	M（5-6天，v9调整：P2.1从2天上调到3-4天）	Mac，\$0	perplexity涨幅数字产出；跨分布矩阵产出；校准集消融产出
3	P3.0-P3.2 AgentBench-OS +服务层封装（v9新增P3.2）	M（5天，v9从3天上调）	Mac，\$0	15-20任务跑通（含3-5个held-out）；结构化vs自由文本 α 对比数字产出；/generate端点跑通多轮工具调用演示并产出加速后端vs基线的端到端延迟对比表
4	P5.0-P5.1+P5.3自适应控制器（Mac部分）	M（4天）	Mac，\$0	GammaTune复现论文量级提速（3次跑均值±标准差）；波动场景测试产出；熔断器重探测机制实现
5	P4.0-P4.1 batch交叉点（Mac部分）+ P2.4 mlx_lm.awq交叉验证（v9新增）	S（2天）	Mac，\$0	交叉点初步曲线产出；显存占用数据产出；mlx_lm.awq真实Mac AWQ速度数字产出
6	P2.4（云端补全）+P4.2+P5.2（含BnB对照臂）+P5.4（三个基线）云端规模化验证	M（4天，v9调整：新增BnB对照臂+BanditSpec代码运行）	云端，\$30-50	LLM Compressor对比数字；vLLM对比数字；AWQ量化- γ 偏移结论+BnB同源对照结论；HF双基线+BanditSpec代码对比数字；held-out集跑一次最终确认
7	产出	S（2天）	\$0	GitHub repo+README定稿；简历bullet定稿

成本注记

Mac阶段全程\$0；云端阶段预算硬顶\$50（护栏指标），超支即停止规模化验证用已有结果收尾，且所有云端脚本必须先本地dry-run（见§10）。总时长：约29-30天（v9从27-28天再次上调，新增原因是P3.2服务层封装+端到端演示）。

§ 12 / VALIDATION PLAN

验证/测试计划

- 贪心模式：逐token严格比对，目标模型直接推理vs投机解码输出必须完全一致（或仅可忽略浮点误差）。
- 采样模式：(a)统计层面验证实测 α 与理论公式 $E[\min(p,q)]$ 吻合；(b)下游任务指标parity（差距<2点）。
- 量化正确性：perplexity涨幅、逐层量化误差分布。
- 自适应控制器：稳定场景+故意设计的波动场景双重测试。
- 验证器自身：故障注入测试（9.6风险3）——手动破坏正确性，确认验证器能抓到。
- 引用溯源：任何写进最终报告的文献数字，附上“论文名+章节/表号+是否为该论文自己的结果”三项标注（9.6风险5）。

- 对标方法论前提核对（v9新增）：任何“对标XX论文数字”的结论，先确认自己的实验设置和对方论文是否可比（9.6风险6），不可比时补同源对照组或改写预期。
- 交叉验证：mlx-lm跑同一模型对做 α 数量级对拍，mlx-lm.awq做AWQ速度交叉验证；参考romsto/Speculative-Decoding等开源实现做输出合理性sanity check；BanditSpec公开代码作为P5.4的第三个对比基线。

§13 / END-TO-END WALKTHROUGH

端到端走读（计划中的产出格式，非已实测数字）

对标参考文档§8“一天的旅程”。参考文档的故事里包含一次险些失败又恢复的交换过程，用叙事本身检验系统在不顺利情况下是否仍然可靠，而非只演示顺利路径。下面的走读同样包含一次“控制器一度做出错误决策、随后依靠自身补救机制恢复”的场景。

一个agent收到任务：“读取repo/utils.py，把其中的parse_config函数重构成支持嵌套key的版本，跑通已有单测。”

第一幕：顺利的主路径

1.

草稿模型先开口。3B级别的草稿模型看到任务描述+文件内容，自回归生成前 $\gamma=5$ 个候选token（比如先吐出def parse_config()）。因为这是代码补全场景（经验依据来自IBM/PyTorch生产复盘：代码场景可以用更多draft token），P5.0的GammaTune控制器此时观察到最近几轮的高接受率，已经把 γ 从默认值扩到7。

2.

目标模型单次前向验证。7B目标模型对这7个候选token做一次前向，逐位比较：前5个匹配，第6个不匹配——接受5个，从目标模型的调整分布里重采样第6个，丢弃第7个候选，本轮产出6个token。P1.3的统计验证器把这一步的接受数记录进 α 的滚动窗口。

3.

量化在背后悄悄生效。目标模型本身是P2阶段自己校准出来的4-bit AWQ版本——如果这一步是在Mac上跑，验证的是“输出和FP16目标模型比perplexity涨幅是否可控”；如果是云端阶段用LLM Compressor的GPTQ版本做对照，验证的是“真实加速比”。

第二幕：批量压力下控制器暂时判断失误——以及它怎么发现自己错了

4.

batch从1涨到16。同时有16个类似的重构任务在跑。P4的batch扫描告诉我们：从某个batch值开始，草稿模型的开销不再被目标模型省下来的前向次数覆盖，P5.3的熔断器观察到吞吐下降信号，把 γ 降到1（等效关闭投机解码）。

5.

与此同时，其中4个任务的性质发生了变化——它们不再是“重构一个函数”这种代码模式规律的任务，而是转成了“读一段自然语言changelog，判断要不要触发一次版本号升级”这种更接近自由文本推理的任务。如果熔断器是DSD那种“关了就不会再看”的设计（坑11），它会持续认为“投机解码在这批任务上不划算”，但这4个任务的 α 下降原因其实与“高batch”无关，是任务类型变了才需要重新评估。这正是9.6风险4想防的陷阱：一个表面符合预期的判断（高batch→该关）掩盖了另一个原因（任务类型变了）。

6.

P5.3设计的周期性重探测机制在第50步插入一次 $\gamma=3$ 的试探——探测到这4个任务的 α 回升到了0.6以上，原因是它们混进了原本就该受益的代码模式任务，与batch下降无关。按照P5.3的设计意图，控制器不会整体重新开启投机解码，而会把粒度下放到per-task级别记录这次探测结果，为后续同类任务重新评估 γ 提供依据。这一行为在实现前属于设计待验证的假设，而非已确认的行为：P5.1的波动场景测试专门用来检验控制器在“部分任务性质变化、部分没变”这种混合场景下是否按预期反应，还是像GammaTune论文承认的那样在非稳态场景下打折扣（坑9）。

第三幕：落到评测

7.

最终评测。这个重构任务本身来自P3改编的AgentBench-OS任务集里的一条，最终是否FINISHED（单测通过）决定了这条trace在下游任务指标里怎么计分。第二幕的那次“熔断器差点判断失误”不会出现在最终的加速比数字里——它会被记录进P5.3的实验日志（附录C），作为“控制器设计是否达到预期”这一独立问题的证据，和“这个任务本身有没有做完”分开评估，不混为一谈。

这条走读连接了支柱1（接受率）、支柱2（量化正确性）、支柱4（batch行为）、支柱5（自适应控制，包括它可能犯错的地方）、支柱3（任务结果）。第二幕的设计是刻意的：只呈现成功场景的系统设计，通常说明设计者没有充分考虑失败模式。

§14 / THE STORY

我们能讲的故事

一句话定位候选（面试/简历场景下用）：

- 1.投机解码和量化是同一个系统原理的两个例证，我测出了这个原理失效的具体边界在哪。
- 2.调库打开投机解码只要一行flag；我把这一行flag背后的正确性证明、失效条件、自适应控制全部手写实现并验证了一遍。
- 3.我知道生产界的最优解是训练专用speculative head——我诚实地选择了预算约束下可行性更高的方案，并且把这个取舍写进了报告里。

北极星论点

这个项目要证明的是对这些加速技巧失效条件的理解，以及设计一个能感知自身失效边界的系统的能力——这代表从“实现论文里的技巧”到“理解系统边界”的差距，也是新grad项目和有2-3年经验的工程师项目之间常见的差距。

§15 / DELIVERABLES

产出物

- GitHub repo（代码+README，含论文引用、prior art引用、诚实边界说明——包括平台兼容性踩坑过程本身）
- 最小服务层封装（P3.2，v9新增）：一个可被外部agent循环调用的/generate端点+一份多轮工具调用场景下加速后端vs基线的端到端延迟对比演示，是repo里唯一“可以直接跑起来看效果”而非只能读评测数字的部分
- 简历bullet（见附录D）
- 本系列计划文档（v1-v9）作为项目思考过程的书面记录，可选择性放入repo的docs/目录

§16 / OPEN QUESTIONS

悬而未决的问题

- 1.具体draft/target模型尺寸对——暂定Qwen2.5系列，等P1.0的a-gate结果才能定。
- 2. P5.3熔断器是否和P4的batch扫描代码共享同一套实现——倾向共享，等P4实现完看接口是否好复用。
- 3.云端预算在阶段5和阶段6之间的具体分配——等实际云端GPU租用单价和阶段4本地测试结果出来后细化；v9新增的BnB对照臂+BanditSpec代码运行会分走一部分预算，需要在阶段6开始前重新估算。
- 4. AgentBench任务集改编的具体任务数量（15个还是20个，扣除held-out后实际可调参的任务数会更少）——先按20个设计，如果太重再削减。
- 5. P5.2的AWQ-BnB对照实验如果测出偏移量级差异很大，报告里要花多少篇幅讨论“为什么不同”（v9新增）——这本身可能是一个值得深入的小发现，但也可能超出项目原定的时间预算，需要在跑出结果后再决定深挖还是简单记录。
- 6. P3.2的服务层是否需要引入JSON schema/正则文法约束解码（v9二次评审新增）——目前P3.2只是测接受率差异，不强制输出必须匹配某个schema，所以不受影响；但如果后续想让/generate端点真正保证输出合法JSON（更贴近生产agent服务的实际需求），需要注意投机解码+grammar约束解码组合使用时存在token被静默污染的已知风险，先记录，暂不在当前预算内处理。

§17 / DECISION LOG

决策记录

v1-v6：均由用户在迭代过程中通过继续深化调研要求的方式确认，尚无正式的“批准进入实现阶段”信号。v7：读完参考文档全部28页后补齐颗粒度缺口，内容决策不变，等待用户确认后再讨论是否进入阶段0。v8：精读5篇自适应控制论文原文并更正一处错误引用（SpecKV “41.2%”），重新措辞BanditSpec不复现的理由；架构/技术决策不变。

V9（本版本）

用户要求“以资深架构师视角，从可行性和已有成熟方案角度”评审v8。评审发现P5.2实验设计存在AWQ/BnB方法论错配风险（已修正为方向性验证+BnB同源对照）、P2.1（AWQ）工作量估计偏乐观（2天→3-4天，总时长25-26天→27-28天）、以及mlx_lm.awq/BanditSpec公开代码/HF双重内置基线三处“已有成熟方案未被利用”的信息差（均已补入相应章节）；§8补充self-speculative作为此前遗漏的备选方案并写明拒绝理由；新增云端预算的本地dry-run执行纪律。架构方向不变，仍等待用户确认后再讨论是否进入阶段0。

V9二次评审（同版本内追加）

用户明确项目定位是SWE/Agent交叉岗位的求职作品集，不是研究产出，据此重新评审了整个文献对比部分：追加下载并精读QSpec/Synergy/SpecKV同类交叉点实验/Learning to Draft/Lossless but Not Free共5篇论文，发现支柱4（AWQ×batch交叉点）和P5.3（batch-aware熔断器）与QSpec、Nightjar在方法/机制层面重叠程度高于此前判断，遂将§2.1/§2.2/§6/P5.3/坑11/附录D的措辞统一从“填补空白”改写为“在自己的约束下复现验证”；同时修正GammaTune一处沿用多版本的作者引用错误（“Kim et al.”应为“Gautam et al.”，经读取论文原文首页确认）；新增P3.2最小服务层封装+agent循环端到端延迟演示，作为向“系统集成能力”而非“研究新颖性”倾斜的具体产出，总时长相应从27-28天上调到29-30天。架构方向不变。

V9三次评审（同版本内再追加：逐条挑战核查）

对全篇可核查的事实性主张做逐条上网/查一手源核实（GitHub API直查issue原文与评论、官方博客原文比对），10项里8项完全通过（含“batch>64开始吞吐下降”这类原以为像编造的精确数字，追一手源后证实是PyTorch官方博客原话）。发现并修正2处问题：§9.1风险A引用的“vLLM CPU模式3-5 tokens/s vs llama.cpp Metal 92 tokens/s”具体数字经查不在标注来源（Issue #1441正文及115条评论）里，属引用溯源错误，已改为不依赖精确数字的定性表述并修正附录E来源条目；vllm-metal“早期版本”的框定已不合时宜（2026年内多次迭代，已用上M5 NAX张量单元），已补充时效性说明。新增一条悬而未决问题：投机解码与JSON schema约束解码组合使用的潜在风险（P3.2当前设计不受影响，仅记录）。架构方向不变。

附录A / FORMULAS & ALGORITHMS

附录A：核心公式、算法伪代码与数值算例

A.1 Rejection Sampling (Leviathan et al. 2023)

```

REFERENCE
for i in 1..γ:
  draft_token[i] ~ p_DM(x | context, draft_token[1..i-1])
  target_probs = TargetModel.forward(context, draft_token[1..γ]) # 单次前向
  n_accepted = 0
  for i in 1..γ:
    r ~ Uniform(0,1)
    if r < min(1, p_TM(draft_token[i]) / p_DM(draft_token[i])):
      accept draft_token[i]; n_accepted += 1
    else:
      break
  if n_accepted < γ:
    x_new ~ norm(max(0, p_TM(x) - p_DM(x))) # 从调整分布重采样
  else:
    x_new ~ p_TM(x | context, draft_token[1..γ]) # bonus token, 必须来自目标模型

```

数值算例 ($\gamma=3$, 词表简化成3个候选token {A, B, C}, 模拟一次verification) : 草稿模型对第1个位置的分布 $p_{DM} = \{A:0.7, B:0.2, C:0.1\}$, 采样得到A。目标模型对同一位置的分布 $p_{TM} = \{A:0.4, B:0.3, C:0.3\}$ 。接受概率 $= \min(1, p_{TM}(A)/p_{DM}(A)) = \min(1, 0.4/0.7) = 0.571$ 。假设采样到的 $r=0.8 > 0.571 \rightarrow$ 拒绝, 本轮到此为止, 不再看后续候选。从调整分布重采样: $p'_{TM} = \text{norm}(\max(0, p_{TM} - p_{DM})) = \text{norm}(\{A:\max(0, 0.4-0.7)=0, B:\max(0, 0.3-0.2)=0.1, C:\max(0, 0.3-0.1)=0.2\}) = \{A:0, B:0.33, C:0.67\}$ 。最终这一步吐出的token从{B:0.33, C:0.67}里采样, 绝不会是A——这符合直觉: 目标模型没那么偏爱A (0.4 vs 草稿模型的0.7), 所以拒绝后重采样把A的概率清零, 公平地把概率让给目标模型相对更青睐、又被草稿模型低估的B和C。

这个算例也直接演示了坑2要防的错误: 如果重采样时不小心又从 p_{DM} 采样 (而不是从上面算出的调整分布 p'_{TM} 采样), 会系统性地偏向草稿模型的偏好 (A), 累积下来的输出分布就不再等于目标模型单独推理的分布——这是“看起来能跑、但正确性保证已经被破坏”的那类bug, P1.2的贪心验证器要能抓到, 9.6风险3要求故意注入这类bug测试验证器本身。

A.2 GammaTune成本模型与Algorithm 1 (Gautam et al. 2025)

```

REFERENCE
cost = N / (αγ+1) × (c+γ) × T_draft, 其中c = T_target / T_draft

if A == γ: # 上一轮全部接受, 可能还有余量
  γ ← A + δ
else:
  γ⁻ ← clip(γ_min, γ_max, (1-η)· γ⁻ + η· A)
  γ ← ceil(γ⁻)

```

三轮数值算例 ($\eta=0.3, \delta=2, \gamma_{\min}=1, \gamma_{\max}=10$, 初始 $\gamma^-=3, \gamma=3$) :

轮次	实际接受数A	触发分支	计算	新 γ^-	新 γ
1	3（全部接受， $A=\gamma$ ）	扩窗	$\gamma \leftarrow 3+2$	（不更新 γ^- ）	5
2	2（部分接受， $A<\gamma$ ）	EMA	$\gamma^- \leftarrow \text{clip}(1, 10, 0.7 \times 3 + 0.3 \times 2) = \text{clip}(1, 10, 2.7)$	2.7	$\text{ceil}(2.7)=3$
3	3（全部接受， $A=\gamma$ ）	扩窗	$\gamma \leftarrow 3+2$	（不更新 γ^- ）	5

只要连续命中“全部接受”， γ 会持续扩窗式增长（3→5→7→...直到 γ_{max} ）；一旦某一轮没有全部接受，立刻回落到EMA的保守估计，不会让 γ 在一次不理想的输出后继续野蛛式增长——这是“扩窗要快，收缩要稳”的设计意图在数字上的体现，也是P5.1波动场景测试要验证的核心行为。原论文还提出了一个GammaTune+变体，额外加入基于草稿模型top-1 logit概率的提前停止阈值 τ ，在4组模型对上平均提速从GammaTune的 $1.15 \pm 0.05x$ 略微提升到 $1.16 \pm 0.03x$ ——Specter不实现这个变体（不在P5.0范围内），只作为“同一套EMA思路还能怎么扩展”的背景参考。

A.3 AWQ逐通道缩放——直觉性数值算例

AWQ的核心洞察：并非所有权重通道都同等重要，激活值幅度大的输入通道对应的权重通道更“显著”，量化时应该被优先保护。一个简化的两通道玩具例子：假设某一层有两个输入通道，激活值统计幅度分别为 $|X1|=10$, $|X2|=1$ （通道1的激活值普遍大10倍，说明这个通道承载的信号更重要）；原始权重 $W1=0.5$, $W2=0.5$ （两个通道权重原本一样大）。

- 如果直接对 $W1$, $W2$ 统一做4-bit量化而不做任何缩放，两个通道的量化误差量级相近——但通道1的误差会被激活值放大10倍（误差 $\times |X1|$ ），对最终输出的影响远大于通道2。
- AWQ的做法：引入缩放因子 s （通常与激活值幅度正相关，比如 $s=|X1|^\alpha$ 的形式， α 是一个校准出来的超参数），量化前把 $W1$ 放大为 $W1 \times s$ 、把对应的激活值缩小为 $X1/s$ ，使数学上等价（ $(W1 \times s) \times (X1/s) = W1 \times X1$ 不变），但量化误差被压缩到激活值更“迟钝”的那一侧。
- 净效果：通道1（重要通道）的有效量化精度因缩放因子而提高，通道2（不重要通道）相应地牺牲一点精度——这组 s 值需要在P2.1中通过校准集统计量计算得出。v9提醒：这个玩具例子只涉及一个数学步骤；P2.1实现时最容易卡住的是fake-quantize的round→clamp→dequantize顺序和激活值统计hook的挂载时机，这两处debug往往比写这个数学步骤本身耗时得多。

这个算例只是帮助建立直觉，P2.1实现时的具体缩放因子搜索方式（网格搜索最小化量化后输出误差）以AWQ原论文Section 3的公式为准，不是这里简化的启发式。

附录B / TASK LIST

附录B：AgentBench-OS子集任务清单（草案，实现时细化）

改编自AgentBench的OS/bash子环境，聚焦“生成结构化输出/工具调用”这一投机解码效果最好的场景，草案任务类别（每类2-4个具体任务，共15-20个，其中3-5个在设计阶段就标记为held-out，仅在超参数定稿后跑一次，见9.6风险1）：

- 1.文件操作类（读取/修改/重命名指定文件）
- 2.代码重构类（如§13走读示例：重构函数签名并跑通单测）
- 3.命令行工具调用类（用grep/find/awk完成指定查询并输出结构化结果）
- 4.多步骤依赖类（前一步的输出是后一步的输入，测试长上下文下的接受率变化）

每个任务的完成判据（FINISHED）：产出的文件/命令输出通过预先写好的校验脚本，不依赖人工判断。

附录C / TELEMETRY & DIAGNOSTICS

附录C：实验日志/ Telemetry Schema + 诊断分析代码

对标参考文档里引用自己产品日志（tx_demand_search_log等）和附录A“只读查询”的做法。Specter没有生产数据库，日志落地为本地文件（jsonl/parquet），下面的“查询”是pandas风格的伪代码，不是真实SQL。

C1.日志字段定义

每个投机解码步骤应记录

step_id, draft_tokens[], target_logits_hash, n_accepted, gamma_used, latency_ms, model_pair_id, quantization_level, quantization_method（v9新增quantization_method字段，用于区分AWQ和BnB对照臂，避免P5.2分析时把两种压缩方式的数据混在一起算）

每个 γ 调整决策应记录（P5.0-P5.3）

timestamp, prev_gamma, new_gamma, trigger_reason（EMA更新/扩窗/熔断/重探测）, acceptance_history_window

每个batch扫描数据点应记录（P4）

batch_size, throughput_tokens_per_sec, draft_model_memory_mb, kv_cache_memory_mb, speedup_ratio_vs_no_spec

每个量化校准实验应记录（P2.2/P2.3）

calibration_dataset, eval_dataset, n_calibration_samples, perplexity, group_size, quantization_method

C2.诊断分析代码（只读，实现阶段直接复用）

REFERENCE

```
#滚动窗口接受率 $\alpha$ （对应P1.3、P5.0的EMA输入）
df_steps["alpha_rolling"] = (
    df_steps["n_accepted"] / df_steps["gamma_used"]
).rolling(window=50).mean()

#加速比按batch_size分布（对应P4，定位交叉点）
speedup_by_batch = (
    df_batch.groupby("batch_size")["speedup_ratio_vs_no_spec"]
    .agg(["mean", "std", "count"])
)
#交叉点= speedup_ratio均值首次跌破1.0对应的batch_size

# $\gamma$ 调整触发原因分布（对应P5.3，检查熔断/重探测是否按预期触发）
trigger_counts = df_gamma_decisions["trigger_reason"].value_counts(normalize=True)

#量化方法对最优 $\gamma$ 的偏移（对应P5.2，坑10核心实验，按quantization_method分开算）
optimal_gamma_by_quant = (
    df_steps.groupby(["quantization_method", "quantization_level"])
    .apply(lambda g: g.loc[g["latency_ms"].idxmin(), "gamma_used"])
)
shift_ratio_by_method = (
    optimal_gamma_by_quant.groupby("quantization_method")
    .agg(lambda s: s.max() / s.min())
)
#预期：shift_ratio_by_method["bitsandbytes"]应接近SpecKV的4倍（同源对照）
# shift_ratio_by_method["awq"]可能明显不同——这本身是可报告的发现，不是异常

#跨分布校准perplexity矩阵（对应P2.2，坑5核心实验）
ppl_matrix = df_calibration.pivot_table(
    index="calibration_dataset", columns="eval_dataset", values="perplexity"
)

#验证器故障注入测试结果（对应9.6风险3）：注入已知bug后必须全部FAIL
assert df_fault_injection_tests["detected"].all(), "验证器漏检——先修验证器再信任其他结果"
```

附录D / RESUME DRAFT

附录D：简历bullet草稿

- 手写实现投机解码采样算法，贪心模式下逐token精确验证正确性，采样模式下通过统计检验和任务指标parity确认分布等价；诊断并规避tokenizer不一致、bonus token误采样等已知实现陷阱
- 自研AWQ风格激活感知量化校准pipeline，通过跨分布校准实验验证其相对GPTQ的抗过拟合优势
- 实测投机解码与量化在不同batch size下的吞吐增益曲线，定位出两者从“内存带宽优化”转为“计算瓶颈拖累”的交叉点，并验证不同量化方法（AWQ vs BitsAndBytes）对最优投机解码步长的系统性影响存在差异
- 实现训练-free的自适应投机解码步长控制器（基于EMA的GammaTune算法），复现Nightjar的batch-aware熔断设计思路（用周期性定时器替代其MAB规划器），对标Hugging Face Transformers双重内置基线及BanditSpec开源实现
- 基于AgentBench方法论改编agent工具调用评测集，验证结构化输出场景下投机解码接受率显著高于自由文本生成
- 封装最小HTTP服务层，将加速后端暴露为可被agent循环直接调用的统一接口，跑通多轮工具调用任务的端到端延迟对比演示（加速后端vs基线）

附录E / REFERENCES

附录E：分类文献与信息来源

一、投机解码核心与SOTA方法

Leviathan et al. 2023 (arXiv 2211.17192, ICML'23) Fast Inference from Transformers via Speculative Decoding
奠基论文之一，rejection sampling正确性证明的来源，附录A.1算例直接基于其算法描述。

Chen et al. 2023 (arXiv 2302.01318, DeepMind) Accelerating LLM Decoding with Speculative Sampling
另一篇奠基论文，70B Chinchilla上2-2.5x加速的实证参考。

Li et al. 2024 (arXiv 2401.15077) EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty
SOTA方案，特征级drafting而非独立小模型，Specter不复现但作为“生产界最优解”在§8/§14里对照引用。

Li et al. 2024 (arXiv 2406.16858) EAGLE-2: Faster Inference with Dynamic Draft Trees
树形草稿+动态调整，接受率更高，SGLang生产数据（batch=24掉速/batch=64仍有效）来自这条线。

Cai et al. 2024 (arXiv 2401.10774) Medusa: Simple LLM Inference Acceleration with Multiple Decoding Heads
多头并行解码对比基线，需要专门训练，是§8“投机解码架构”决策点的拒绝对象之一。

Meta 2024 (arXiv 2404.16710, ACL'24) LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding
self-speculative路线的代表作（v9新增），训练时对早期层用低dropout、后期层用高dropout+提前退出损失，推理时不需要单独草稿模型，LLaMA上最高2.16x提速；§8已把这条路线列为“考虑过但拒绝”的方案。

综述2025 (arXiv 2502.19732) Speculative Decoding and Beyond: An In-Depth Survey
全景综述，含SpecInfer/Sequoia等树形方法背景。

二、量化方法

Lin et al. 2023 (arXiv 2306.00978) AWQ: Activation-aware Weight Quantization
支柱2的核心方法来源，跨分布perplexity数字（+0.5-0.6）、附录A.3算例均基于此文。

Frantar et al. 2023 (arXiv 2210.17323) GPTQ: Accurate Post-Training Quantization
对比基线，Hessian-based，跨分布perplexity涨幅（+2.3-4.9）数字来源，本身不重新实现（§3非目标1）。

Xiao et al. 2023 (arXiv 2211.10438) SmoothQuant: Accurate and Efficient PTQ for LLMs
W8A8方案，仅作背景对比，说明W4A16（AWQ路线）更适合本项目的低batch/延迟优先场景。

Dettmers et al. BitsAndBytes
NF4/INT8量化，HF Transformers内置（load_in_4bit=True），v9新增作为P5.2的同源对照臂，直接复现SpecKV原始压缩设置，不算重新实现GPTQ。

QSpec 2024 (arXiv 2410.11305) Speculative Decoding with Complementary Quantization Schemes
v9二次评审新增：Figure 7已经画出AWQ vs FP16在batch=8/16/32下投机解码加速比收窄的曲线，早支柱4两年发表，本项目原文献综述漏检了这篇。支柱4的定位相应调整为“在自己的模型对/硬件约束下复现同类交叉点”，不再声称是没人测过的实验（见§2.2、§6）。

Speculative Decoding Meets Quantization 2025 (arXiv 2505.22179)
另一篇测投机解码和量化方案（W8A8/W4A16/W4A8-QQQ）兼容性的论文，Llama-3-70B+EAGLE-2，支柱2/AWQ这条线上的又一个先例，v9二次评审新增。

Lossless but Not Free 2026 (arXiv 2607.17283) An Empirical Anatomy of Speculative Decoding on Consumer Hardware
v9二次评审新增：从内存带宽/计算瓶颈切换的机制层面解释了为什么AWQ（带宽优化）会和batch size在投机解码里产生交叉效应——batch=1时算术强度远低于计算/带宽平衡点、大batch把weight读取摊薄到多个流上，逐步转向计算瓶颈，给支柱4的现象提供更底层的机制依据。

三、服务层架构

Kwon et al. 2023 (arXiv 2309.06180, SOSP'23) Efficient Memory Management for LLM Serving with PagedAttention
vLLM底层架构论文，决定“造轮子vs用轮子”边界：Specter不重新实现KV cache管理，vLLM云端阶段作为现成对比基线引入。

The Synergy of Speculative Decoding and Batching 2023 (arXiv 2310.18813)
v9二次评审新增：最早系统提出batch=1时投机解码降63%延迟、batch=32时应缩短y这一内存带宽↔计算瓶颈切换机制的论文，支柱4的理论基础其实这篇已经讲清楚，本项目做的是在自己的模型对上实测复现这条曲线，不是发现这个机制。

四、Agent评测方法论

Liu et al. 2023 (arXiv 2308.03688, ICLR'24) AgentBench: Evaluating LLMs as Agents

支柱3任务设计方法论来源，只改编其OS/bash子环境 (§3非目标2)。

Qin et al. 2023 (arXiv 2307.16789) ToolLLM: Facilitating LLMs to Master 16000+ APIs

工具调用任务集设计参考。

五、自适应投机步长控制（2024-2026前沿，七篇均已精读原文）

Gautam et al. 2025 (arXiv 2504.00030, Texas A&M) Token-Driven GammaTune

P5.0核心算法来源，Algorithm 1 (EMA更新+扩窗) 直接实现，成本模型中 $c=T_{\text{target}}/T_{\text{draft}}$ 典型取值4-10；4组模型对上平均提速 $1.15\pm0.05x$ ，单张80GB H100实测；GammaTune+变体 (logit提前停止) 提速略升至 $1.16\pm0.03x$ (附录A.2已引，不实现)；

Limitations章节明确承认低方差场景收益有限、对抗性场景会退化 (坑9)。v9补充：2026年新出现的RL-based自适应drafting工作 (Learning to Draft, 见后) 已经把GammaTune当作要超越的baseline在用，说明这类训练-free EMA启发式到2026年中已经不是这个方向的前沿——选它是预算约束下的合理取舍 (不需要训练数据/训练基础设施)，不是贴近SOTA的选择，两者需要分开陈述。

Learning to Draft 2026 (arXiv 2603.01639) Adaptive Speculative Decoding with Reinforcement Learning

v9新增，直接回应上一条GammaTune条目末尾的提醒：这篇论文把GammaTune列为要超越的baseline之一，用RL训练的策略网络在线选择 γ ，在其报告的设置下超过GammaTune等训练-free启发式方法。收入的目的不是打算实现RL控制器 (超出§3非目标的预算边界)，而是如实标注GammaTune在2026年中的文献坐标——本项目选训练-free方案是可解释性和零训练成本的工程取舍，不是因为它是当前最优。

SpecKV 2026 (arXiv 2605.02888) Adaptive Speculative Decoding with Compression-Aware Gamma Selection

最优 γ 随压缩程度 (BitsAndBytes的FP16/INT8/NF4) 从2偏移到8 (4倍/300%) (Table 1)；headline结果是其MLP-16控制器让每步期望token数比固定 $\gamma=4$ 的基线提升56.0%，控制器开销仅0.34ms。v9提醒：这个实验用的压缩方式是BnB，不是AWQ，P5.2对标这个数字时需要注意方法论前提是否一致 (见坑10、9.6风险6)。

Agrawal et al. 2024 (arXiv 2410.18351, Qualcomm AI Research) AdaEDL: Early Draft Stopping via Entropy-based Lower Bound

熵基提前停止判据 $1 - \sqrt{(\gamma \cdot H_{DM}(x))} < \lambda$ 来源，注意此处 γ 是论文自定义的“熵因子”超参数 (全部实验固定为0.2)，和投机解码通常语境下的“步长 γ ”是两个不同的量，命名冲突。DL=3/7/16下接受token数标准差递增 (1.2/1.92/2.35) 确认来源为其Figure 1/附录Fig 7c。独立佐证坑4的数据点：未微调的TinyLlama-1B给Llama2-7B做草稿、固定投机长度=7时，静态投机解码反而比自回归基线慢16%，自适应提前停止后变成快43%。只作文献对比不重新实现 (§3非目标3)。

BanditSpec 2025 (arXiv 2505.15141) Adaptive Speculative Decoding via Bandit Algorithms

UCB (UCBSpec) /EXP3 (EXP3Spec) 双算法框架，论文自己在附录B.2明确说UCBSpec “是最简单的一类UCB算法之一”；附录B.4明确将自己定位为与SpecDec++ “正交”的方案——训练-free vs训练-based。v9新增：本论文有公开代码

github.com/sail-sg/BanditSpec (兼容LLaMA/Qwen2架构)，P5.4直接克隆运行作为第三个对比基线，不修改其算法逻辑。

Nightjar 2025 (arXiv 2512.22420) Dynamic Adaptive Speculative Decoding for LLM Serving

高负载30.25%吞吐倒退的数字来源 (坑7，原文Section 3.1)；批评DSD “重激活难题”和BanditSpec “忽视切换开销” (坑11)；自己实测的KV cache切换开销 (Table 3, RTX 4090 + DeepSeek-R1-Distill-Qwen-7B) 在17.87ms到102.03ms区间；headline结果是6个7B/13B基准设置上平均比无投机解码提速27.29%、比标准投机解码提速8.32% (最高14.76%)、比BanditSpec/DSD分别提速22.89%/19.76%。

Lu, Hong, Liu et al. 2025-2026 (arXiv 2512.11280) AdaSD: Adaptive Speculative Decoding for Efficient Language Model Inference

v9新增，评审时发现的更新文献。双组件 (熵判定终止生成+ Jensen-Shannon距离判定接受标准)、超参数-free的自适应方案，在Qwen系列模型上有实验 (Table 2)，最高1.46x提速、精度损失<1.8%。不涉及量化维度，和P5.2/SpecKV不重叠，只作为“自适应投机解码是活跃前沿方向”的又一佐证收入文献综述。

六、生产实践/工程博客 (“理论之后发生了什么”)

A Hitchhiker's Guide to Speculative Decoding (PyTorch官方博客，2024，IBM Research生产环境实测)

全项目最重要的一手资料：Llama2-13B/Llama3-8B/Granite-7B 2x提速，Granite-20B代码模型3x提速；代码模型用更多draft token更划算 (P1.1/P3设计依据)；batch>64开始吞吐下降 (支柱4存在理由)；训练speculative head比经典两模型方案效果更好但需要多卡训练资源 (§8架构决策拒绝理由的一手证据)；精确对比应用贪心而非采样 (P1.2设计依据)；先测接受率再测吞吐的方法论 (P1.0 gate设计依据)。

HF blog: Dynamic Speculation Lookahead (v9新增)

HF Transformers内置的assistant_confidence_threshold机制的方法论来源，Intel labs与HF合作，最高2.7x提速；P5.4现在明确把这个机制和更简单的num_assistant_tokens_schedule="heuristic"分开对待，因为前者方法论上更接近AdaEDL的熵/置信度思路，是更强的基线。

HF docs: Assisted Decoding (v9新增)

num_assistant_tokens_schedule和assistant_confidence_threshold两个参数的官方文档，P5.4两个基线的具体配置依据。

vLLM Blog: Speculative Decoding

原生支持EAGLE/EAGLE-3/Medusa/n-gram草稿，一个flag开启，佐证“调库无技术含量”论点（\$2.2）。

SGLang文档

EAGLE在batch=24掉速、EAGLE-3在batch=64仍有效的具体数字来源（支柱4）。

llama.cpp docs/speculative.md

经典两模型方案+n-gram自投机的现状参考，EAGLE-3支持仍在讨论（Issue #15902）。

AutoAWQ GitHub

官方文档承认高batch场景下W4A16因反量化开销可能变慢（坑8/支柱4呼应），且明确CUDA-only（风险B来源之一）。

mlx-lm GitHub (v9新增)

原生支持mlx_lm.awq，Mac端真实AWQ量化+Metal加速推理，P2.4交叉验证来源，也是P1.0/P4的 α 与速度sanity check工具。

Medium: A practical guide to INT4 quantization for SLMs

GPTQ vs AWQ真实场景对比，W4A16延迟优先/W8A8吞吐优先的场景划分依据。

LLM Compressor (vLLM团队)

统一GPTQ/AWQ/FP8量化API，行业从分裂工具链（AutoGPTQ+AutoAWQ）走向统一工具的现状，决定P2.4对比基线工具选择（\$8）。

vLLM Issue #1441

标题"Does vllm support the Mac/Metal/MPS?"，正文实测报错"Cannot find CUDA_HOME"，vLLM在Mac上原生不可用的直接证据（风险A）。v9二次核查：这个issue只能证明“不可用”这个定性结论，不含任何tokens/s性能数字，此前版本曾把第三方测评文章里的具体数字挂在这个issue名下，已在正文改为不依赖精确数字的定性表述。

vllm-metal GitHub (v9更新时效性说明)

官方README显示2026年内进展较快：v0.2.0起统一Metal kernel成为默认attention backend（较v0.1.0提速83x TTFT/3.6x吞吐），2026-08已用上M5 NAX张量单元。仍是MLX技术栈而非vLLM官方CUDA路径，不能替代云端阶段的vLLM基线，但“早期版本、模型覆盖不全”这个框定需要承认它在持续快速迭代。

AutoGPTQ Issue #223

AutoGPTQ需要专门MPS kernel（未实现）的维护者原话（风险B）。

bitsandbytes MPS backend PR #1853 (v9新增)

佐证PyTorch量化生态对MPS支持普遍不成熟这一现状（风险B），同时说明BnB的MPS支持目前处于PR阶段未必稳定，P5.2的BnB对照臂建议仍放在云端CUDA环境跑，不依赖这个未合并的PR。

七、参考开源实现（对拍用，不直接抄）

romsto/Speculative-Decoding

干净实现Leviathan et al. 2023算法，含经典自回归/beam search/投机解码三种对照，用于P1.2输出合理性sanity check。

foundation-model-stack/fms-extras

IBM生产代码开源部分，展示PagedAttention kernel如何支持多head KV cache管理，帮助理解投机解码+KV cache管理结合的工程难点。

Pramodith Dissects博客+notebook

SmolLM2-360M/1.7B手写贪心投机解码，模型选择思路（同系列不同尺寸）与Qwen2.5系列思路一致， α 值/加速比量级sanity check参考。

sail-sg/BanditSpec (v9新增)

BanditSpec论文官方代码，P5.4直接克隆运行作为第三个对比基线，见\$7/坑12。